

PearlPath

Centralized digital travel platform

Developer Guide

CO2060 — Second Year Project

Department of Computer Engineering

University of Peradeniya

Team

E/22/124 — R.G. Gunawardana

E/22/228 — W.M.S. Manujitha

E/22/452 — Y.M.C.J. Yagabamunu

E/22/232 — D.F.A.T.D. Mathangadeera

Contents

1	Introduction	2
2	System Overview	2
3	System Architecture	2
4	Technology Stack	3
5	Project Structure	3
6	Prerequisites	4
7	Installation and Setup	4
8	Environment Variables	4
9	Running the Application	5
10	Database Design	5
10.1	User (models/User.js)	5
10.2	Booking (models/Booking.js)	5
10.3	Other Key Models:	5
11	API Documentation	6
12	Authentication and Authorization	6
13	Feature Implementation	7
13.1	Hotel Search & Discovery	7
13.2	Route Planning & Maps	7
13.3	AI Assistant (Trip Planner)	7
14	External APIs and Services	7
15	Error Handling	7
16	Security and Validation	8
17	Testing	8
18	Git and GitHub Workflow	8
19	Common Issues and Troubleshooting	9
20	Deployment	9
21	Future Development	9
22	Developer Contribution Guide	10

1 Introduction

Welcome to the Developer Guide for **PearlPath**. This guide is written for future developers who need to understand, run, maintain, and extend the system.

PearlPath is a centralized digital travel support platform designed to simplify travel planning in Sri Lanka. It connects tourists with verified service providers including attractions, hotels, tour guides, and vehicle services into a single, cohesive system.

- **Target Users:** Tourists (domestic & international), Service Providers (Hotel Owners, Vehicle Owners, Tour Guides), and System Administrators.
- **Main Features:** Hotel/Vehicle/Guide search & booking, Route planning with maps, AI-based travel assistant, and Currency conversion.
- **Technologies Used:** MERN Stack (MongoDB, Express.js, React.js, Node.js).

2 System Overview

PearlPath unifies multiple disconnected travel services. Tourists visiting Sri Lanka often face a fragmented digital ecosystem where accommodation, transportation, and tour services are spread across multiple platforms. This leads to inefficiencies and difficulty in planning a reliable travel experience.

With PearlPath, a tourist can search for a destination, find nearby hotels, book a tour guide, and rent a vehicle all in one flow. The backend handles the business logic, secure bookings, and external integrations (like email notifications and AI processing), while the frontend delivers a highly interactive, responsive user interface.

3 System Architecture

PearlPath follows a scalable, standard three-tier architecture:

```
User
|
v
React.js Frontend (Vite)
|
v
Node.js + Express Backend
|
v
MongoDB Database (Mongoose)

External Services:
|-- Leaflet / OpenStreetMap (Maps & Routing)
|-- Nodemailer (Emails)
|-- OpenAI / Gemini API (AI Assistant)
```

4 Technology Stack

Component	Technology
Frontend	React.js 19 (Vite)
Styling	Tailwind CSS v4, SweetAlert2, Lucide React
Backend	Node.js + Express.js v5
Database	MongoDB + Mongoose
Maps & Routing	Leaflet + React-Leaflet
Email	Nodemailer
AI Assistant	OpenAI API / Google Generative AI
Authentication	JWT (JSON Web Tokens) + bcryptjs
Version Control	Git + GitHub

5 Project Structure

```
PearlPath/
|-- code/
|   |-- frontend/
|   |   |-- public/           # Static assets
|   |   |-- src/
|   |   |   |-- components/  # Reusable UI components
|   |   |   |-- pages/       # Route-level components
|   |   |   |-- assets/      # Images and CSS files
|   |   |   |-- App.jsx      # Main application router
|   |   |   |-- main.jsx     # React entry point
|   |   |-- package.json
|   |   |-- vite.config.js   # Vite configuration
|   |
|   |-- backend/
|   |   |-- config/           # Configurations (e.g., payments)
|   |   |-- controllers/     # API business logic
|   |   |-- middlewares/     # Express middlewares (Auth, Multer)
|   |   |-- models/          # MongoDB Schemas (User, Hotel, etc.)
|   |   |-- routers/         # API Route definitions
|   |   |-- services/        # External services (AI integration)
|   |   |-- scripts/         # DB seed scripts
|   |   |-- uploads/         # Local file storage
|   |   |-- server.js        # Node.js server entry point
|   |   |-- package.json
|   |
|-- docs/                    # Project documentation
|-- README.md
```

6 Prerequisites

To run this project locally, ensure you have installed:

- **Node.js** (v18+ recommended)
- **npm** (comes with Node)
- **MongoDB** (Local instance or a MongoDB Atlas account)
- **Git**
- **Code Editor** (VS Code recommended)
- **API Keys** (Email App Passwords, OpenAI/Gemini keys)

7 Installation and Setup

1. Clone the repository:

```
git clone <repository-url>
cd e22-co2060-PearlPath
```

2. Frontend Setup:

```
cd code/frontend
npm install
```

3. Backend Setup:

```
cd ../backend
npm install
```

8 Environment Variables

Create `.env` files in the respective directories. **Never commit actual `.env` files to version control.**

Backend `.env` (code/backend/.env):

```
PORT=3001
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret_key
EMAIL_HOST=smtplib.org
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_specific_password
EMAIL_FROM="PearlPath Support" <your_email@gmail.com>
GEMINI_API_KEY=your_gemini_api_key
OPENAI_API_KEY=your_openai_api_key
```

Frontend `.env` (code/frontend/.env) (Optional, for API routing):

```
VITE_API_BASE_URL=http://localhost:3001/api
```

9 Running the Application

Open two separate terminals for the Frontend and Backend.

Terminal 1 (Backend):

```
cd code/backend
node server.js
```

(Runs by default on `http://localhost:3001`)

Terminal 2 (Frontend):

```
cd code/frontend
npm run dev
```

(Runs by default on Vite's port `http://localhost:5173`)

10 Database Design

The database uses MongoDB via Mongoose. The core collections include:

10.1 User (`models/User.js`)

Field	Type	Description
name	String	User's full name
email	String	User's email address
password	String	Encrypted password
role	String	tourist / admin / hotel_owner / etc.

10.2 Booking (`models/Booking.js`)

Field	Type	Description
user	ObjectId	Reference to the user who booked
itemType	String	Hotel, Vehicle, TourGuide
itemId	ObjectId	Reference to the booked service
date	Date	Date of booking
status	String	pending, confirmed, cancelled

10.3 Other Key Models:

- `Hotel.js`, `Vehicle.js`, `TourGuide.js`: Service provider listings.
- `Experience.js`: Tourist attractions and destinations.
- `Review.js`: User ratings and feedback.
- `Payment.js`: Transaction tracking.

11 API Documentation

The backend exposes RESTful endpoints prefixed with `/api`. Below are examples of important endpoints:

Method	Endpoint	Purpose	Auth
POST	<code>/api/users/register</code>	Register a new user	No
POST	<code>/api/users/login</code>	Authenticate and get JWT	No
GET	<code>/api/hotels</code>	Fetch all approved hotels	No
POST	<code>/api/bookings</code>	Create a new reservation	Yes
GET	<code>/api/bookings/my-bookings</code>	Get bookings for logged-in user	Yes
POST	<code>/api/trip-planner/plan</code>	Generate AI itinerary	Yes

Example Payload (Login):

```
POST /api/users/login
Request Body:
{
  "email": "tourist@example.com",
  "password": "mySecurePassword"
}
Response:
{
  "token": "eyJhbG...",
  "user": { "id": "...", "role": "tourist" }
}
```

12 Authentication and Authorization

- **Registration/Login:** Users register and log in to receive a JWT token. Passwords are securely hashed using `bcryptjs`.
- **JWT Middleware:** Protected routes require an `Authorization: Bearer <token>` header. The backend middleware verifies this token.
- **Role-Based Access Control (RBAC):**
 - **Tourist:** Can search, book, and review.
 - **Provider (Hotel/Vehicle/Guide):** Can manage their specific listings and view incoming bookings.
 - **Admin:** Has overarching control to approve listings, view system stats, and manage users.

13 Feature Implementation

13.1 Hotel Search & Discovery

```
User applies filters on Frontend
      |
Frontend calls GET /api/hotels?location=Kandy&price=low
      |
Backend executes MongoDB query/aggregation
      |
Results returned to Frontend and rendered dynamically
```

13.2 Route Planning & Maps

Uses Leaflet to render maps. Locations (Lat/Lng) fetched from the backend are plotted as markers.

13.3 AI Assistant (Trip Planner)

The `/api/trip-planner` endpoint takes user parameters (days, budget, interests), securely constructs a prompt, and queries the OpenAI/Gemini API, returning a parsed day-by-day itinerary.

14 External APIs and Services

- **Leaflet / React-Leaflet:** Open-source mapping library used to display interactive maps and markers without requiring paid API keys.
- **Nodemailer:** Used in the backend to send automated email notifications (e.g., OTPs, booking confirmations) via SMTP (like Gmail).
- **OpenAI / Google Generative AI:** Processes natural language inputs to power the chatbot and dynamic trip generation features.
- **Multer:** Node.js middleware for handling `multipart/form-data` (uploading images to the local `/uploads` directory).

15 Error Handling

Standardized API error handling flow:

```
Frontend request -> Backend Validation -> If Invalid -> Return Error
```

Common HTTP Status Codes used:

- 200 OK – Request succeeded.
- 201 Created – Resource successfully created.
- 400 Bad Request – Missing or invalid fields.
- 401 Unauthorized – Invalid/Missing JWT token.
- 403 Forbidden – User lacks correct role permissions.
- 404 Not Found – Route or resource does not exist.
- 500 Server Error – Internal backend crash or DB failure.

16 Security and Validation

- **Input Validation:** Backend controllers validate request bodies before processing (e.g., preventing empty emails).
- **Password Hashing:** `bcryptjs` ensures passwords are never saved in plain text.
- **API Key Protection:** External keys (Gemini, OpenAI, Email) are strictly kept in the backend `.env`.
- **CORS Configuration:** The `cors()` middleware in `server.js` protects the API from unauthorized cross-origin requests.
- **Payload Limits:** `express.json({ limit: '50mb' })` prevents massive payload denial-of-service.

17 Testing

Currently, testing is heavily manual and API-driven:

Feature	Testing Tool	Test Scope
Backend APIs	Postman	Verify 200/400/500 responses and payloads
UI/UX Flow	Browser	Verify React states, routing, and alerts
Auth Flow	Postman/UI	Validate JWT creation and expiry
Image Upload	Postman/UI	Validate multipart parsing and storage

Developers should ensure all core routes (Auth, Booking, Search) return 200 OK in Postman before submitting PRs.

18 Git and GitHub Workflow

```
main
|-- feature/add-payment-gateway
|-- feature/ui-enhancements
'-- bugfix/map-rendering-error
```

1. Always pull the latest main branch: `git pull origin main`.
2. Create a specific branch for your work: `git checkout -b feature/your-feature-name`.
3. Write clear commit messages.
4. Push your branch and open a Pull Request against `main`.

19 Common Issues and Troubleshooting

Problem	Possible Solution
MongoDB connection failed	Ensure your IP is whitelisted in MongoDB Atlas or check <code>MONGO_URI</code> .
CORS Errors in browser	Verify frontend URL matches the origins allowed in backend <code>server.js</code> .
Token Expired / Unauthorized	Clear local storage in browser and login again.
Email Not Sending	Use a Gmail "App Password", not your standard account password.
Vite / Node Modules Error	Delete <code>node_modules</code> and <code>package-lock.json</code> , then run <code>npm install</code> .

20 Deployment

(Currently configured for local development. Future deployment steps:)

1. **Frontend (Vercel/Netlify):** Add environment variables pointing to the production backend URL and deploy the `dist/` folder generated by `npm run build`.
2. **Backend (Render/Heroku/AWS):** Deploy the Node server, ensuring `MONGO_URI` and all secret keys are added to the hosting provider's Environment Variables panel.
3. **Database (MongoDB Atlas):** Update Network Access to whitelist the deployed backend's IP address (or allow `0.0.0.0/0` temporarily).

21 Future Development

Future developers may look to extend the platform with:

- **Full Online Payments Integration:** Expanding `paymentRouter` with Stripe or PayHere for live transactions.
- **Real-Time GPS Tracking:** WebSockets to track vehicle rentals.
- **Cloud Object Storage:** Migrating local Multer uploads to AWS S3 or Cloudinary.
- **Automated Refund System:** Processing cancellations securely.

22 Developer Contribution Guide

To contribute to PearlPath:

1. Clone the repository and set up `.env` files.
2. Create a new branch: `git checkout -b feature/new-idea`.
3. Implement the feature. Ensure code is linted (`npm run lint` on frontend).
4. Test locally using Postman and the browser.
5. Commit changes with a descriptive message.
6. Push your branch: `git push origin feature/new-idea`.
7. Create a Pull Request and await code review before merging into `main`.